

Introducción a la computación paralela

Agenda

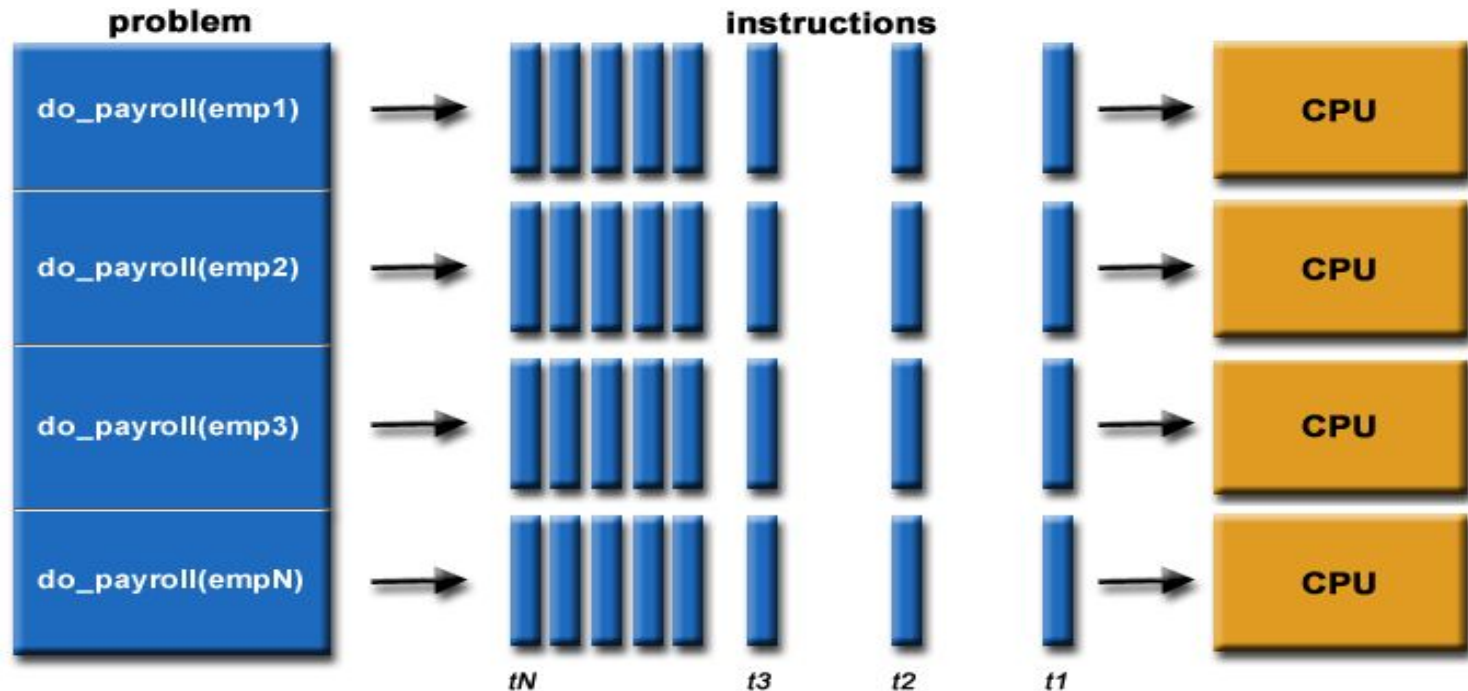
1. Definición de computación paralela
2. Aplicaciones de la computación paralela
3. Clasificación de máquinas paralelas
 - a. Taxonomía de Flynn
 - b. Memoria compartida vs. Distribuida
4. Modelos de programación paralela

Definición de Computación Paralela

Definición de Computación Paralela

- Ian Foster: “***A parallel computer is a set of processors that are able to work cooperatively to solve a computational problem***”
- La **Computación Paralela** es el uso simultáneo de **múltiples recursos de cómputo** para resolver un **problema computacional**:
 - El problema es dividido en partes que pueden resolverse concurrentemente.
 - Instrucciones de cada parte se ejecutan simultáneamente en diferentes procesadores.
 - Debe emplearse un mecanismo de coordinación general.

Definición de Computación Paralela



Computación de Alto Rendimiento High Performance Computing (HPC)

Es el uso de un sistema de cómputo de alto rendimiento para ejecutar aplicaciones de alta demanda computacional de manera rápida. HPC hace posible la ejecución de cálculos previamente inviables a través de una colección de potentes:

- Sistemas de hardware
- Herramientas de software
- Modelos de programación paralela
- Paradigmas de programación paralela



Concurrente, Paralelo, Distribuido

- En la computación **concurrente**, múltiples tareas están en progreso en cualquier momento.
- En el cómputo **paralelo**, múltiples tareas ejecutan de manera simultánea y estrecha para resolver cooperativamente un problema. Todo programa paralelo es concurrente.
- La distinción entre cómputo paralelo y distribuido no es tan clara, aunque normalmente se considera que:
 - “**paralelo**” se relaciona más a una ejecución en una máquina fuertemente acoplada (memoria compartida o memoria distribuida con una red de alto rendimiento), y programas contruidos para ejecutarse muy eficientemente.
 - “**distribuido**” se relaciona más a la ejecución en una máquina débilmente acoplada (red de comunicaciones lenta y normalmente no

Aplicaciones de la computación paralela

... en dispositivos cotidianos

- **Equipos:** teléfonos inteligentes, tablets, notebooks, computadoras de escritorio.



- **Motivación:** Experiencia de Usuario (UX) y Eficiencia Energética.

Lógica: uso paralelismo porque quiero fluidez sin que el dispositivo se caliente o apague cuando depende de una batería.

- **Ejemplo:** Grabar video 4K sin que el teléfono arda en tu mano o agote su batería en pocos segundos.

... en sistemas ciber-físicos

- **Equipos:**

- Robots
- Drones
- Equipamiento médico de soporte a la vida
- Sistemas Avanzados de Asistencia al Conductor

- **Motivación:** Ejecución en tiempo real.

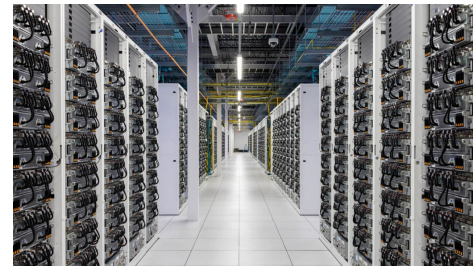
Lógica: uso paralelismo porque la respuesta tiene que ser inmediata para ser segura.

- **Ejemplo:** Para implementar asistencia de dirección automática, el auto debe analizar videos obtenidos con cámaras, datos de distancia y velocidad medidos por radar (radio) y LiDAR (luz), y datos dinámicos del vehículo.



... en centros de datos

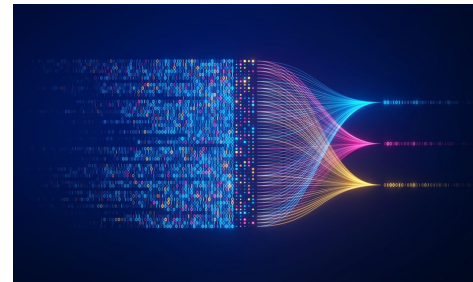
- **Equipos:** servidores interconectados que trabajan cooperativamente.



- **Motivación:** procesamiento masivo de datos.

Lógica: uso paralelismo porque el procesamiento de un gran volumen de datos excede la capacidad del procesamiento serie.

- **Ejemplo:** En la indexación de la Web se requieren procesar petabytes de información repartiendo los datos entre miles de nodos para obtener resultados en tiempos viables.

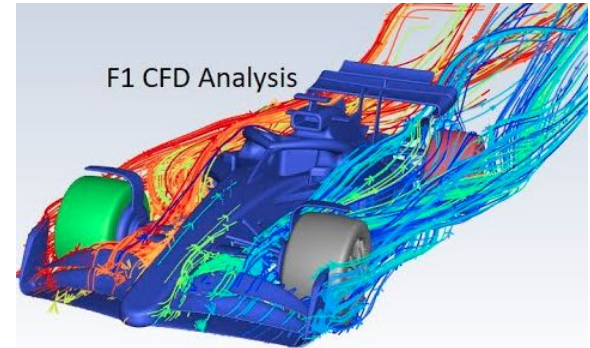


... en sistemas de HPC

- **Equipos:** Nodos de cómputo de alto rendimiento con interconexión de alta velocidad.
- **Motivación:** Acelerar el cómputo intensivo.

Lógica: uso paralelismo para dar mejores soluciones o abordar nuevos problemas que resultan inviables con cómputo secuencial.

- **Ejemplo:** La simulación de dinámica de fluidos permite el diseño aerodinámico de un coche de Fórmula 1.



... en sistemas de HPC: Supercomputación

- Una nueva supercomputadora y las investigaciones que se podrán llevar a cabo.
https://youtu.be/ci_fV_uH-s0 <https://youtu.be/Pgy7Jjybl00>
- Alya Red: HPC-based Computational Biomechanics for Supercomputers.
<https://youtu.be/gaG21uZrZ3E>

¿Es posible hacerlo sin cómputo paralelo?

- Intervalo entre latidos del corazón: de 60 a 200 pulsaciones por minuto, es decir, una pulsación cada 1.000 o 300 ms.
- ¿Cuánto se tarda aproximadamente en simular un latido del corazón?
 - ✧ Con 200 cores del Marenosturm: 100 minutos.
 - ✧ Con 1 core: _____.
 - ✧ Con 10.000 cores del Marenosturm: _____.
- ¿A cuánto equivale 1 día de simulación en Marenosturm?
 - ✧ En un día y con 10.000 cores se pueden simular ____ latidos.
 - ✧ Usando un solo core, el tiempo requerido es de _____.

... en sistemas de HPC: Supercomputación

- Una nueva supercomputadora y las investigaciones que se podrán llevar a cabo.
https://youtu.be/ci_fV_uH-s0 <https://youtu.be/Pgy7Jjybl00>
- Alya Red: HPC-based Computational Biomechanics for Supercomputers.
<https://youtu.be/gaG21uZrZ3E>

¿Es posible hacerlo sin cómputo paralelo?

- Intervalo entre latidos del corazón: de 60 a 200 pulsaciones por minuto, es decir, una pulsación cada 1.000 o 300 ms.
- ¿Cuánto se tarda aproximadamente en simular un latido del corazón?
 - ✧ Con 200 cores del Marenostrium: 100 minutos.
 - ✧ Con 1 core: aprox. 200 veces más, es decir, 20.000 minutos = 333,33 h = 13,88 días.
 - ✧ Con 10.000 cores del Marenostrium: 2 minutos.
- ¿A cuánto equivale 1 día de simulación en Marenostrium?
 - ✧ En un día y con 10.000 cores se pueden simular 720 latidos.
 - ✧ Usando un solo core, el tiempo requerido es de 10.000 días o 27,4 años.

Clasificación de máquinas paralelas

Clasificación de máquinas paralelas

Para utilizar correctamente un sistema de HPC es imprescindible conocer la **arquitectura** del hardware.

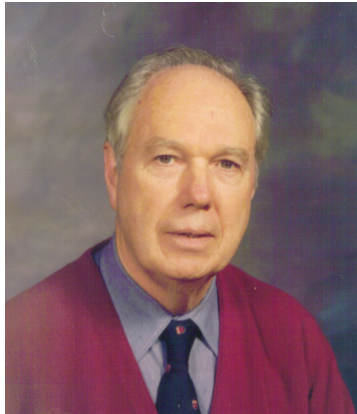
Dos de las clasificaciones más utilizadas son:

- Taxonomía de Flynn (1966)
- Memoria compartida vs. distribuida

Taxonomía de Flynn

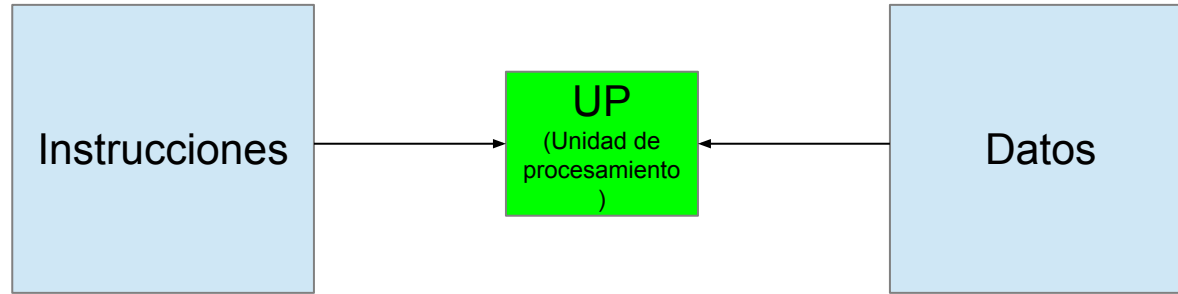
Define cuatro tipos diferentes de arquitecturas de computadoras en función de si hay múltiples flujos de datos y/o múltiples flujos de instrucciones.

Michael J. Flynn

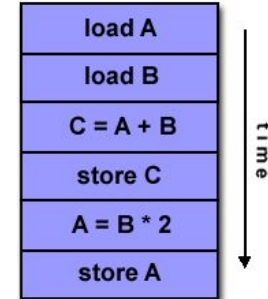


S I S D Single Instruction Stream Single Data Stream	S I M D Single Instruction Stream Multiple Data Stream
M I S D Multiple Instruction Stream Single Data Stream	M I M D Multiple Instruction Stream Multiple Data Stream

SISD

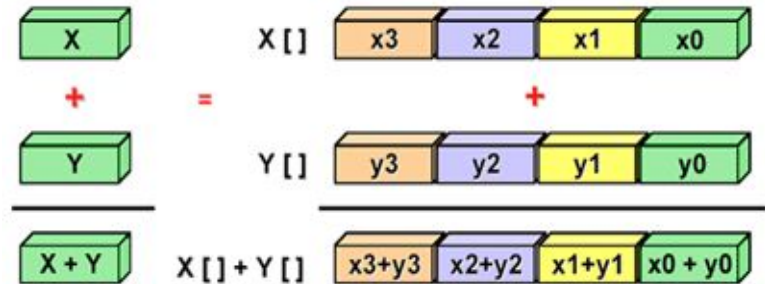
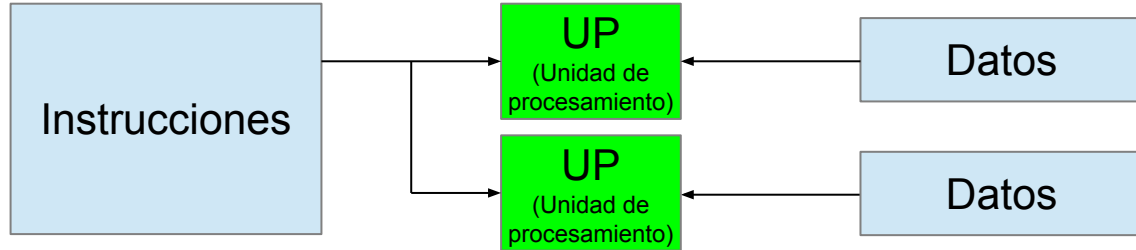
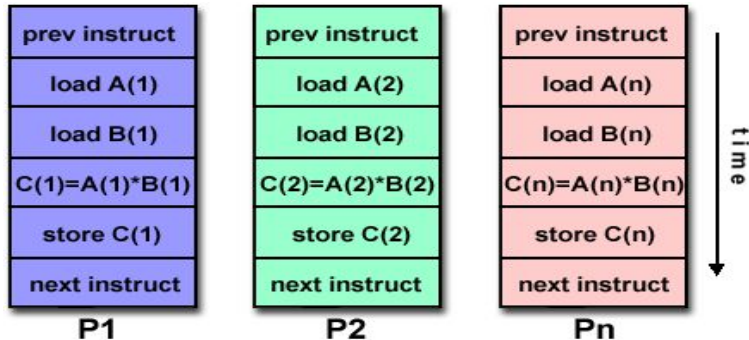


- **Single Instruction Stream:** Un único flujo de instrucciones
- **Single Data Stream:** Un único flujo de datos



SIMD

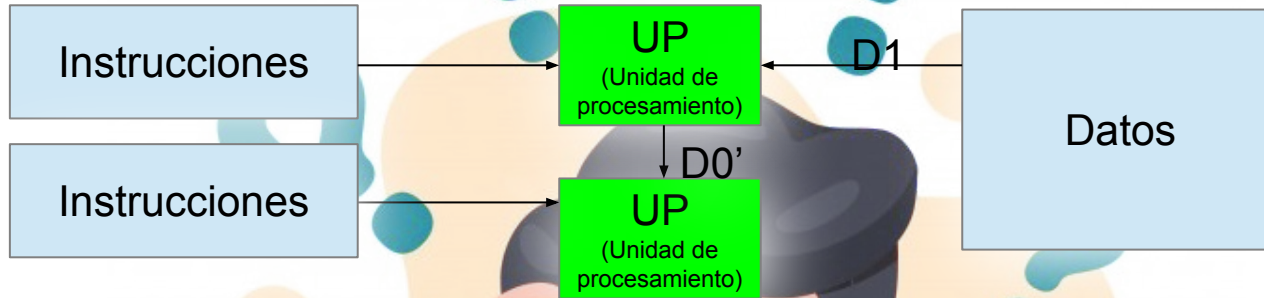
- **Single Instruction Stream:** Un único flujo de instrucciones.
- **Multiple Data Stream:** Cada unidad de procesamiento puede operar en diferentes elementos de datos.



Dos variedades: Array Processor y Vector Processor.

Actuales: GPUs y CPUs con instrucciones vectoriales.

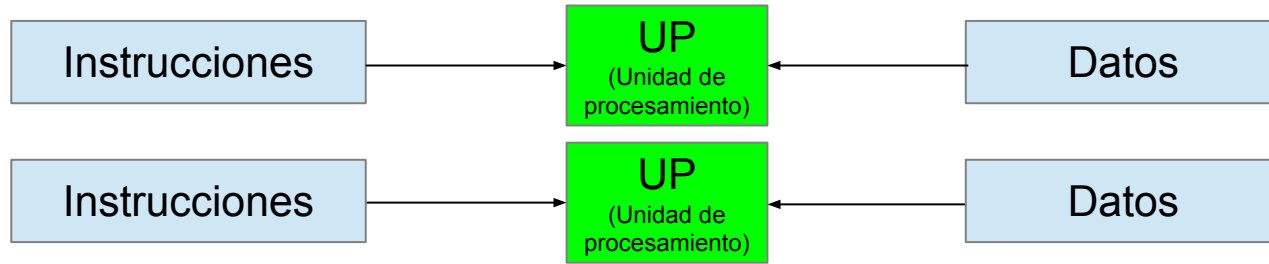
MISD



- **Multiple Instruction Stream:** Cada unidad de procesamiento opera sobre los datos de manera independiente a través de distintos flujos de instrucciones.
- **Single Data Stream:** Un único flujo de datos se introduce en una unidad de procesamiento, y el resultado pasa a la siguiente.

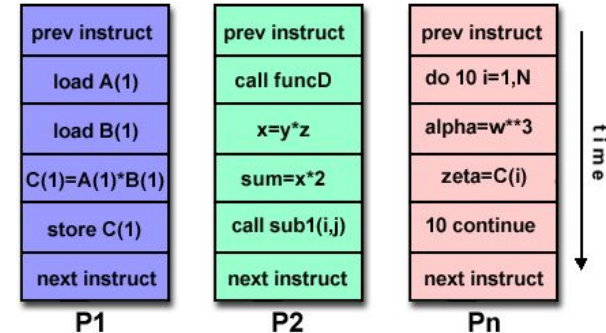
MIMD

- **Multiple Instruction Stream:** Cada unidad de procesamiento puede ejecutar un flujo diferente de instrucciones.
- **Multiple Data Stream:** Cada unidad de procesamiento puede trabajar con un flujo de datos diferente.



Un procesador multicore es MIMD. Incluso, la mayoría de los modernos sistemas de cómputo paralelo caen en esta categoría.

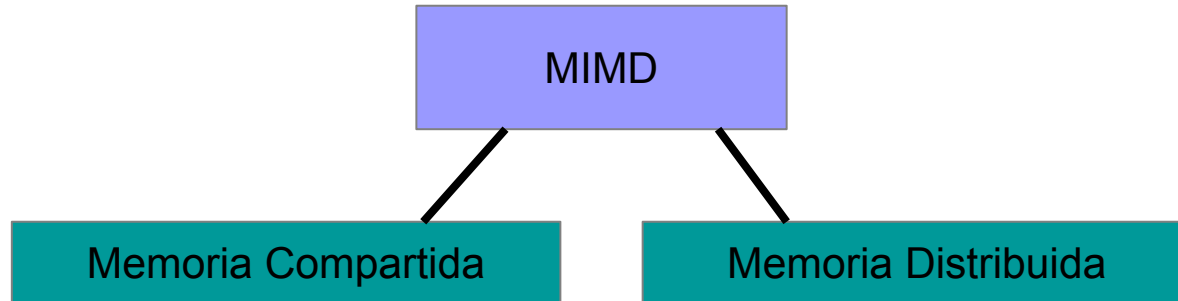
Muchos sistemas MIMD también incluyen subcomponentes de ejecución SISD y SIMD.



MIMD

Memoria compartida vs. distribuida

Esta clasificación diferencia dos tipos de sistemas MIMD, basándose en la visión que el procesador tiene de la memoria principal.



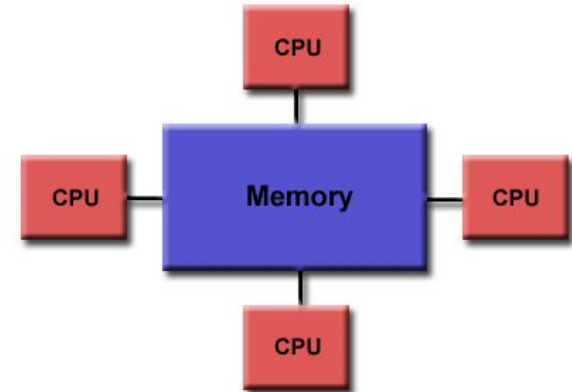
MIMD

Memoria compartida

Estos sistemas pueden tener distinta arquitectura, pero tienen en común que:

Cada uno de los procesadores pueden acceder a toda la memoria del sistema a través de un espacio de direcciones global.

- Múltiples procesadores pueden funcionar de forma independiente, pero comparten los mismos recursos de memoria.
- Los cambios en una ubicación de memoria efectuado por un procesador son visibles para todos los demás procesadores.

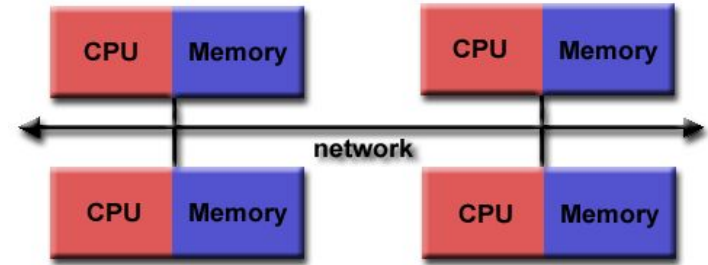


MIMD

Memoria compartida

También son conocidas como **máquinas de paso de mensajes** (*message passing machines*)

- Los procesadores tienen su propia memoria local y no hay un espacio de direcciones global
- Se requiere una red de interconexión para comunicar a los procesadores. Hay varias tecnologías de red disponibles y distintas topologías a considerar
- Para que un procesador lea un dato producido por otro procesador, es necesario que se comunique a través de la red.



Modelos de programación paralela

Modelos de programación

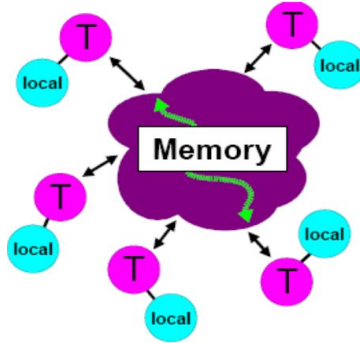
Un modelo de programación describe la visión que el programador tiene sobre la máquina paralela. Esta especificación determina cómo el programador puede codificar un algoritmo.

Hay dos modelos de programación principales, que en teoría pueden ser implementados en cualquier hardware subyacente. Ellos son:

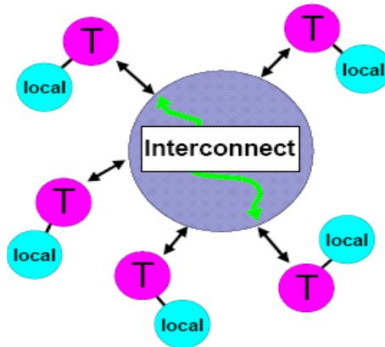
- Modelo de programación de memoria compartida
- Modelo de programación de memoria distribuida

Modelos de programación

- Modelo de memoria compartida



- Modelo de memoria distribuida (paso de mensajes)



Modelos de programación

- Aunque estos dos modelos están inspirados en su correspondiente arquitectura de sistema paralelo, es posible tener:
 - Un modelo de memoria compartida sobre una máquina de memoria distribuida.
 - Un modelo de memoria distribuida sobre una máquina de memoria compartida.
- El uso de un modelo de programación en una máquina que no lo soporta nativamente posiblemente resulte en menor rendimiento.
- **OpenMP** es el modelo de memoria compartida más utilizado. OpenMP no funciona en máquinas de memoria distribuida.
 - Intel implementó “Cluster OpenMP” pero está discontinuado.
- **MPI (Message Passing Interface)** es el modelo de memoria distribuida más utilizado. MPI también funciona en máquinas de memoria compartida.

Bibliografía

- Del libro de Dongarra:
 - 1.2 What Have We Learned from Applications?
 - 1.3 Software and Algorithms
 - 9.1.1 Parallel Programming Models: Shared Memory versus Distributed Memory
- Del libro de Grama:
 - 1.2 Scope of Parallel Computing
 - 2.3 Dichotomy of Parallel Computing Platforms
- Del libro de Pacheco:
 - 1.1 Why We Need Ever-Increasing Performance
 - 1.6 Concurrent, Parallel, Distributed
- Parallel architectures. Michael Flynn, Kevin Rudd. ACM Computing Surveys. 1996.
 - <http://www.di.unipi.it/~vannesch/SPA/p67-flynn.pdf>